

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 0603

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)*

Assessment Tool Weightage: 10%

QUESTIONS: 10

Total Marks:20

Annexure A

Mock Interview Questions

Code of Conduct:

I Mohammed Mudhassir A/192424367(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Mudhassir

1. Hospital Patient Data Cleaning

I would use **Pandas** for DataFrame operations and **NumPy** for fast numerical processing.

Steps:

1. Read the CSV using `pd.read_csv()`.
2. Check missing values using `isnull().sum()`.
3. Fill numerical missing values using median and categorical missing values using mode.
4. Remove duplicate records using `drop_duplicates()`.
5. Standardize inconsistent units, such as converting all weights to kg and temperatures to °C.
6. Use Boolean filtering to remove invalid values.
7. Set `Patient_ID` or timestamp as the index for efficient access.
8. Use aggregation functions such as `mean()`, `min()`, `max()`, and `count()` for reporting.
9. Check for anomalies such as impossible age, negative values, or abnormal measurements.
10. Save the cleaned data using `to_csv()`.

```
df = pd.read_csv("patients.csv")
```

```
df["Age"] = df["Age"].fillna(df["Age"].median())
```

```
df = df.drop_duplicates()
```

```
df = df[(df["Age"] >= 0) & (df["Age"] <= 120)]
```

```
df["Gender"] = df["Gender"].str.strip().str.upper()
```

```
summary = df.groupby("Department")["Age"].mean()
```

```
df.to_csv("cleaned_patients.csv", index=False)
```

For near-real-time processing, files can be processed in **chunks**, and anomalies can be logged instead of stopping the entire pipeline.

2. Large Financial Transaction Dataset

For millions of records, I would use Pandas for structured data and NumPy for numerical calculations.

Approach:

1. Read only required columns.
2. Use appropriate data types such as float32 where precision permits.
3. Process large files in chunks using chunksize.
4. Filter transactions using Boolean indexing.
5. Group transactions by customer or account.
6. Calculate sum, mean, standard deviation, and transaction count.
7. Use NumPy vectorization instead of Python loops.
8. Sort or index frequently searched columns.

```
df = pd.read_csv("transactions.csv",  
                usecols=["Customer_ID", "Amount"],  
                chunksize=100000)
```

for chunk in df:

```
    valid = chunk[chunk["Amount"] > 0]  
    summary = valid.groupby("Customer_ID")["Amount"].sum()
```

Performance can be debugged by checking execution time, memory usage, DataFrame size, and unnecessary copies. Incorrect outputs can be checked using small sample datasets and independent calculations.

3. Combining Sales, Marketing, and Support Data

First, I would read each file according to its format and standardize column names and data types.

```
sales = pd.read_csv("sales.csv")  
marketing = pd.read_excel("marketing.xlsx")  
support = pd.read_json("support.json")
```

Steps:

1. Standardize column names.
2. Convert common keys such as Customer_ID to the same data type.
3. Check missing and duplicate keys.

4. Remove only genuine duplicates.
5. Merge DataFrames using the common key.
6. Use an outer merge when preserving all valid records is important.
7. Group records to identify duplicate customers.
8. Sort the final DataFrame.
9. Reset the index.
10. Validate record counts and missing values.

An outer merge with an indicator can identify unmatched records:

```
result = pd.merge(  
    sales, marketing,  
    on="Customer_ID",  
    how="outer",  
    indicator=True  
)
```

Conflicts can be resolved using source priority, latest valid value, or other predefined business rules.

4. Automated Daily Data Analysis Pipeline

The workflow would contain **file loading** → **cleaning** → **filtering** → **aggregation** → **visualization** → **report generation**.

for file in files:

try:

```
df = pd.read_csv(file)
```

```
df = df.dropna()
```

```
df = df[df["Sales"] > 0]
```

```
summary = (
```

```
    df.groupby("Category")["Sales"]
```

```
    .agg(["sum", "mean", "count"])
```

```
    .sort_values("sum", ascending=False)
```

)

```
summary.plot(kind="bar")
```

```
except Exception as e:
```

```
    print("Error:", file, e)
```

For corrupted files, use exception handling. For inconsistent formats, standardize column names and data types. If required columns are missing, log the file and skip it rather than terminating the entire pipeline.

5. E-Commerce Clickstream Preprocessing

I would use Pandas to clean and structure clickstream records.

Steps:

1. Read the clickstream data.
2. Convert timestamps to datetime.
3. Handle missing values.
4. Remove duplicate events.
5. Standardize categorical values.
6. Filter invalid events.
7. Extract useful features such as hour, day, session duration, and number of clicks.
8. Set timestamp or User_ID as an appropriate index depending on the analysis.
9. Encode categorical variables.
10. Normalize numerical features.
11. Split the prepared data into features and target for machine learning.

Example:

```
df["Timestamp"] = pd.to_datetime(df["Timestamp"])
```

```
df["Hour"] = df["Timestamp"].dt.hour
```

```
df["Page"] = df["Page"].str.strip().str.lower()
```

```
df = df.drop_duplicates()
```

```
df = df.dropna(subset=["User_ID"])
```

This produces a clean and structured dataset suitable for machine learning.

6. Healthcare Vital Time-Series Analysis

I would first convert timestamps and sort the data chronologically.

```
df["Timestamp"] = pd.to_datetime(df["Timestamp"])
```

```
df = df.sort_values("Timestamp")
```

```
df = df.set_index("Timestamp")
```

Then I would:

1. Remove invalid readings.
2. Handle missing timestamps.
3. Resample data to a fixed interval.
4. Interpolate missing numerical values.
5. Calculate rolling mean and standard deviation.
6. Detect unusual values using thresholds or Z-scores.
7. Aggregate readings for reporting.

```
df = df.resample("1min").mean()
```

```
df = df.interpolate()
```

```
df["Moving_Average"] = df["Heart_Rate"].rolling(5).mean()
```

```
df["Rolling_STD"] = df["Heart_Rate"].rolling(5).std()
```

Rolling statistics help identify sudden changes or abnormal patterns while aggregation provides useful summaries.

7. Preparing Millions of Mixed-Type Records for ML

The preprocessing pipeline should be optimized to avoid unnecessary copies.

Steps:

1. Identify numerical and categorical columns.
2. Convert numerical columns using `pd.to_numeric()`.
3. Fill missing numerical values using median.
4. Fill categorical missing values using mode or "Unknown".

5. Encode categorical variables using one-hot encoding or label encoding.
6. Normalize numerical features.
7. Convert the final numerical data to NumPy arrays.
8. Use vectorized operations.

```
df["Age"] = pd.to_numeric(df["Age"], errors="coerce")
```

```
df["Age"] = df["Age"].fillna(df["Age"].median())
```

```
df = pd.get_dummies(df, columns=["Gender"])
```

For optimization, use appropriate data types, avoid loops, process data in chunks, and select only required columns.

Debugging involves checking shape, dtype, missing values, duplicate columns, and unexpected encoded values.

8. Logistics Delivery Performance

I would build a Pandas pipeline using **filtering, grouping, aggregation, indexing, and sorting**.

```
valid = df[
    (df["Delivery_Time"] > 0) &
    (df["Distance"] >= 0)
]

summary = (
    valid.groupby("Location")["Delivery_Time"]
    .agg(["mean", "min", "max", "count"])
)
```

I would:

- Set frequently queried columns as indexes where appropriate.
- Sort by timestamp or location.
- Filter invalid delivery records.
- Group by location, traffic condition, or delivery zone.
- Calculate average delivery time and total deliveries.

- Validate totals against the original dataset.

The calculated metrics should be tested using a small sample where the expected values can be calculated manually.

9. Data Validation System

The validation system should check the incoming dataset before it enters the ML pipeline.

Validation rules include:

- Required columns must exist.
- Data types must be correct.
- Missing values must be detected.
- Numerical values must fall within valid ranges.
- Dates must have valid formats.
- Duplicate IDs must be detected.

```
missing = df.isnull().sum()
```

```
duplicates = df.duplicated().sum()
```

```
invalid_age = df[
```

```
    (df["Age"] < 0) | (df["Age"] > 120)
```

```
]
```

Filtering identifies invalid records, indexing allows efficient record lookup, and aggregation provides quality statistics such as missing-value counts and duplicate counts.

A validation report should contain total records, valid records, invalid records, missing values, and constraint violations.

10. Large User Activity Logs

I would use timestamps and user IDs to efficiently process activity logs.

```
df["Timestamp"] = pd.to_datetime(df["Timestamp"])
```

```
df["Hour"] = df["Timestamp"].dt.hour
```

```
peak = (
```

```

df.groupby("Hour")
    .size()
    .sort_values(ascending=False)
)

```

For user segmentation:

```

user_summary = df.groupby("User_ID").agg(
    Sessions=("Session_ID", "nunique"),
    Activities=("User_ID", "count")
)

```

For scalability:

- Process files in chunks.
- Select only required columns.
- Use efficient data types.
- Use vectorized NumPy/Pandas operations.
- Avoid unnecessary DataFrame copies.
- Aggregate data incrementally.

For real-time performance, process only new records rather than repeatedly processing the complete historical dataset.

Simple Questions

11. Cleaning Student Records

For a small dataset, Pandas provides simple and effective operations.

```
df["Marks"] = df["Marks"].fillna(df["Marks"].median())
```

```

df["Grade"] = (
    df["Grade"]
    .str.strip()
    .str.upper()
)

```

```
df["Grade"] = df["Grade"].replace({
```

```
"A+": "A+",  
"A": "A",  
"B+": "B+",  
"B": "B"  
})
```

Steps:

1. Load the dataset.
2. Find missing marks.
3. Fill missing marks using mean or median.
4. Remove spaces from grades.
5. Convert grades to a common format.
6. Check for duplicates and invalid values.
7. Save the cleaned dataset.

12. Total and Average Sales by Category

Use `groupby()` with aggregation functions.

```
result = df.groupby("Category")["Sales"].agg(  
    Total_Sales="sum",  
    Average_Sales="mean"  
)
```

Steps:

1. Load the sales DataFrame.
2. Select the category column.
3. Group records using `groupby("Category")`.
4. Calculate total using `sum()`.
5. Calculate average using `mean()`.
6. Display or save the result.

This efficiently produces the total and average sales for every product category.

13. Duplicate Entries and Incorrect Data Types

First identify duplicates:

```
duplicates = df[df.duplicated()]
```

Remove duplicate records:

```
df = df.drop_duplicates()
```

Convert numbers stored as strings:

```
df["Amount"] = pd.to_numeric(  
    df["Amount"],  
    errors="coerce"  
)
```

Then check for missing values:

```
print(df.isnull().sum())
```

Finally, validate data types and ranges. Invalid records can be filtered or corrected before analysis.

14. Sorting Timestamps and Moving Average

First convert timestamps into datetime format:

```
df["Timestamp"] = pd.to_datetime(df["Timestamp"])
```

Sort the data:

```
df = df.sort_values("Timestamp")
```

Set the timestamp as the index:

```
df = df.set_index("Timestamp")
```

Calculate a simple moving average:

```
df["Moving_Average"] = (  
    df["Value"].rolling(window=3).mean()  
)
```

The rolling(3) operation calculates the average of every three consecutive observations.

15. Basic ML Preprocessing

The preprocessing steps are:

1. Handle Missing Values

```
df["Age"] = df["Age"].fillna(
```

```
df["Age"].median()
)
For categorical data:
df["Gender"] = df["Gender"].fillna(
    df["Gender"].mode()[0]
)
```

2. Encode Categorical Variables

```
df = pd.get_dummies(
    df,
    columns=["Gender", "Department"]
)
```

3. Normalize Numerical Features

Using NumPy:

```
X = df[["Age", "Salary"]].to_numpy()
```

```
X = (X - X.min(axis=0)) / (
    X.max(axis=0) - X.min(axis=0)
)
```

This converts numerical values to approximately the **0–1 range**.

4. Final Validation

Check for remaining missing values, incorrect data types, and invalid values before passing the data to the machine-learning model.